NLP: Mini Project Report

Name:	Sudev Prasad Dahitule
SAP ID:	60004230171
Roll No.:	C-153
Division:	C1

TOPIC:

N-Ticipate, made from scratch, system-wide predictive text engine using an n-gram language model with a Hidden Markov Model (HMM) part-of-speech tagger for grammar-aware reranking, shipped as a Windows tray application.

SOFTWARE/IDE USED:

Visual Studio Code, Python 3.12, Jupyter Notebook, Windows 11

ABSTRACT:

N-Ticipate is a text autocomplete engine built from first principles. A trigram language model, trained on 6.7 million tokens of modern web prose and dialogue, proposes the next word or completes a partly typed one; a hand-written HMM part-of-speech tagger then reranks those candidates so that grammatically plausible words rise to the top. The engine runs inside a Windows system-tray application that watches keystrokes in any program, shows a small suggestion popup near the caret, and types the rest of the word when the user presses Tab. Every component, tokenizer pipeline, smoothing methods, prefix trie, Viterbi decoder, is implemented without an NLP library on the prediction path, and everything runs locally. The final system saves 42.7% of keystrokes in simulation, answers in 2.15 ms at the 95th percentile, and tags English and Hindi at 93.5% and 94.7% accuracy respectively.

INTRODUCTION:

Predictive text is one of the most visible applications of statistical language modelling: every phone keyboard guesses the next word. Underneath, the problem is to estimate P(w | history), the probability of a word given what came before it, quickly enough to answer on every keystroke. N-gram models make this tractable by assuming that only the last n−1 words matter (the Markov assumption), and smoothing techniques redistribute probability mass so that word sequences never seen in training are not assigned zero probability.

A pure n-gram model knows nothing about grammar, however. After "the quick brown", it may rank a verb as highly as a noun if the counts happen to favour it. N-Ticipate adds a second, independent model, an HMM POS tagger decoded with the Viterbi algorithm, and uses the tag-transition probabilities it learns as a grammatical prior over the candidates. The project was built in eight numbered phases, each producing a tested module and measured results, and covers the core topics of an NLP course: text preprocessing, language modelling and smoothing, perplexity, sequence labelling with HMMs, and evaluation on a regional language (Hindi).

NEED OF THE PROJECT:

- **Typing effort:** Long words and frequent phrases ("let me know", "see you tomorrow") are typed thousands of times; a predictor that is right even a third of the time saves a large share of keystrokes.
- **System-wide availability:** Phone keyboards predict everywhere, but on Windows prediction exists only inside individual apps. A global hook brings it to Notepad, browsers and chat clients alike.
- **Privacy:** Cloud-based assistants send what the user types to a server. N-Ticipate runs entirely offline; the context buffer has no save method at all, and learning from the user is opt-in.
- **Accessibility:** Users with motor impairments or slow typing speed benefit most from fewer keystrokes per word.
- **Learning value:** Building every stage from scratch exposes the trade-offs (smoothing vs. pruning, log-space vs. linear-space Viterbi) that a library call would hide.

TECHNICAL STACK:

Table 1: Technologies used and their role

Layer	Technology	Purpose
Language	Python 3.12	Entire engine and desktop app
Numerics	NumPy	HMM transition / emission matrices, vectorised Viterbi
Configuration	YAML (config.yaml)	Single home for every tunable; no magic numbers in code
NLP reference	NLTK	Sentence splitter and an offline correctness cross-check only, never on the prediction path
Training data	HuggingFace datasets	Streams fineweb-edu and dialogsum corpora
Tagged corpora	Universal Dependencies	English-EWT, English-GUM, Hindi-HDTB (.conllu)
Keyboard hook	pynput + Win32 (ctypes)	Global low-level keystroke capture and word injection
User interface	Tkinter, pystray	Never-focusable overlay popup, tray icon, fallback editor
Evaluation	Jupyter, Matplotlib, pandas	Per-phase notebooks, tables and plots
Testing	pytest	609 automated tests across all modules

ARCHITECTURE:

The system is split into an offline training side and a runtime side, connected only through four model files. Training runs once on the developer machine; the runtime loads the files and never touches the raw corpora (Figure 1). Each phase of the project corresponds to exactly one module, which keeps the code testable in isolation (Table 2).

Table 2: Module structure

Module	Phase	Responsibility
preprocess.py	1	Clean, segment, tokenize, build vocabulary, <UNK> rare words, truecase map, train/dev/test split
ngram.py	2	NgramModel, MLE, Laplace, stupid backoff, Kneser-Ney, perplexity, pruning
trie.py, userprofile.py	3	Prefix trie for completion; per-user n-gram model learned from typing

Module	Phase	Responsibility
predictor.py	3, 6	Predictor.predict(), single entry point; optional POS reranking term
hmm.py	4, 5	HMMTagger, transition/emission/initial matrices, log-space Viterbi
app/ (6 files)	7	Keystroke hook, debouncer, overlay, injector, Win32 helpers, tray app
scripts/	8	fetch_data.py downloads corpora; retrain.py rebuilds every artefact

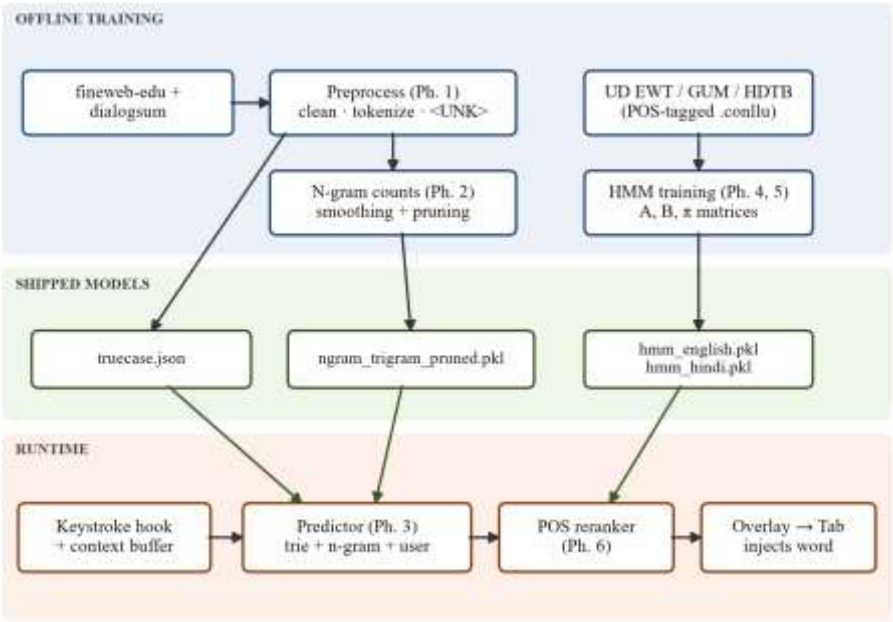


Figure 1: System architecture, training pipeline, shipped models and runtime pipeline

WORKING:

1. Preprocessing

Raw text is cleaned, split into sentences and tokenized. Words seen fewer than three times are replaced with <UNK>, which closes the vocabulary and gives the model a way to score words it has never seen. Unlike in text classification, stopwords and punctuation are kept, "of the" is exactly what a bigram model must learn. Counting is done in lowercase, and a separate truecase map restores natural capitalisation ("internet" → "Internet") when a suggestion is displayed.

2. N-gram Language Model

Unigram, bigram and trigram counts are collected and four estimators are compared: maximum likelihood (which shows the zero-probability problem), Laplace add-k, stupid backoff, and interpolated Kneser-Ney. Quality is measured by perplexity on held-out text:

$$PP(W) = P(w_1\ w_2\ \ldots\ w_N)^{(-1/N)}$$

Kneser-Ney gives the lowest perplexity on the full model, but after pruning rare n-grams to make the file shippable it degrades three times faster than stupid backoff, because its continuation counts come from exactly the rare n-grams that pruning removes. The shipped model is therefore a pruned trigram with stupid backoff:

$$S(w \mid u, v) = c(u\ v\ w) / c(u\ v) \quad \text{if } c(u\ v\ w) > 0, \quad \text{else } 0.4 \cdot S(w \mid v)$$

Table 3: Dev-set perplexity by order and smoothing (unpruned, Brown-era development run)

n	MLE	Laplace	Stupid backoff*	Kneser-Ney
1	700.4	703.5	700.4	700.4
2	∞	1,701.7	222.2	238.4
3	∞	9,704.3	291.3	221.1

** Stupid backoff is unnormalised, so its figure is a ranking score rather than a true perplexity.*

3. Prediction Engine

If the user has typed part of a word, a character-level prefix trie returns every vocabulary word with that prefix ("rec" → "recommend"), and the n-gram model scores them in context. If no prefix has been typed, the model proposes next words directly by backing off from trigram to bigram to unigram. A per-user model learned from the user's own typing is blended in, with λ growing as more user data accumulates; its vocabulary is never <UNK>-ed, so names and jargon are learned too:

$$score(w) = \lambda \cdot P_user(w \mid h) + (1 - \lambda) \cdot P_base(w \mid h)$$

4. HMM Part-of-Speech Tagger (English and Hindi)

The tagger learns three probability tables from a tagged corpus: the initial distribution π, the tag-transition matrix $A = P(t_i \mid t_{i-1})$ and the emission matrix $B = P(w_i \mid t_i)$, all smoothed with add-k (k = 0.01). The most likely tag sequence is decoded with the Viterbi algorithm in log space, since multiplying hundreds of small probabilities underflows to zero in linear space:

$$\delta_t(j) = max_i \left[\delta_{t-1}(i) + log\, A_{ij} \right] + log\, B_j(w_t)$$

Unknown words fall back on suffix heuristics (-ing/-ed → VERB, -ly → ADV, -tion → NOUN). The same class trains on Hindi with no code changes beyond a Devanagari suffix branch and treating the danda (।) as sentence punctuation.

5. POS-aware Reranking

The tagger is attached to the predictor as one extra term. Each candidate's tag is guessed context-free (tagging every candidate in context would cost one Viterbi decode per candidate per keystroke), and its score becomes:

$$score(w) = log\, P(w \mid context) + \alpha \cdot log\, P(tag(w) \mid tag(previous\ word)), \quad \alpha = 0.1$$

With α = 0 the ranking is exactly the plain n-gram ranking, so the feature is safe to leave on.

6. Desktop Application

A global low-level keyboard hook maintains a rolling context buffer (last two words plus the current prefix). A debouncer fires a prediction 50 ms after typing pauses, and a frameless, never-focusable overlay shows the suggestions near the caret. Tab is intercepted inside the hook procedure itself so that Windows never delivers it to the application underneath; the injector then types the rest of the word. Figure 2 shows the per-keystroke flow.

7. Corpus Swap and Retraining

The first version was trained on the 1961 Brown corpus, and every metric was healthy, yet using the app showed suggestions nobody would accept: "can you pl" offered place, play, plan but never "please", and "Internet" was not in the vocabulary. Held-out evaluation on the same corpus cannot detect such a domain mismatch. The corpora were replaced with fineweb-edu and dialogsum for the language model and Universal Dependencies treebanks for the taggers, and every model was retrained without any code change.

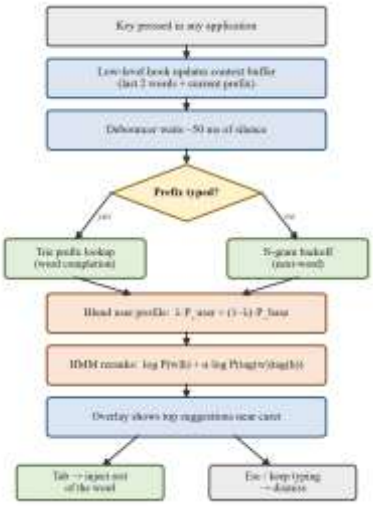


Figure 2: Runtime flow for a single keystroke

RESULTS:

Table 4: Final measured results

Metric	Value	Target / Baseline
Training tokens	6.73 M	927 k (Brown)
Out-of-vocabulary rate (dev)	2.07%	3.68% (Brown)
Next-word hit@1 / hit@3	19.2% / 32.1%	16.7% / 29.0%
Completion hit@1 (2 characters typed)	46.0%	40.2%
Keystroke savings	42.7%	40.4%
p95 prediction latency (completion)	2.15 ms	≤ 50 ms budget
English tagger accuracy	93.48%	88.76% baseline
Hindi tagger accuracy	94.72%	92.26% baseline
Shipped n-gram model size (pruned)	12.5 MB	5.28 M → 0.86 M n-grams
Automated tests	609 passed	1 skipped

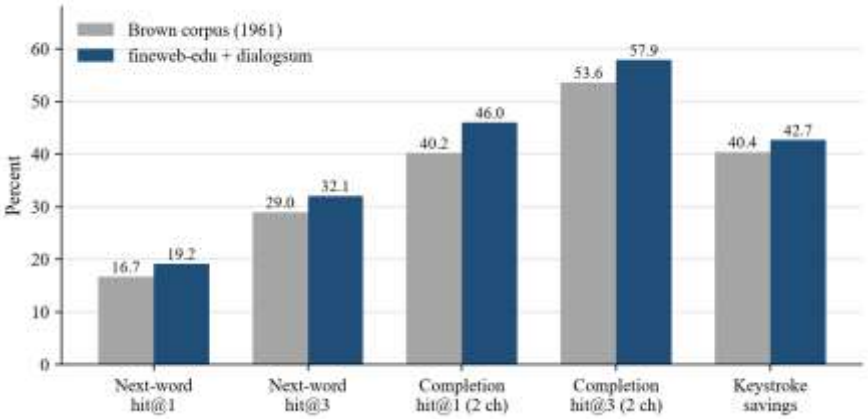


Figure 3: Prediction accuracy and keystroke savings before and after the corpus swap

Hindi scores higher than English, the opposite of what was expected. The explanation is the out-of-vocabulary rate rather than the language: the Hindi treebank is newswire with 2.8% unseen words, while the English treebanks are web text with 3.5%. On the unseen words themselves English is ahead, as its suffixes are more regular (Figure 4).

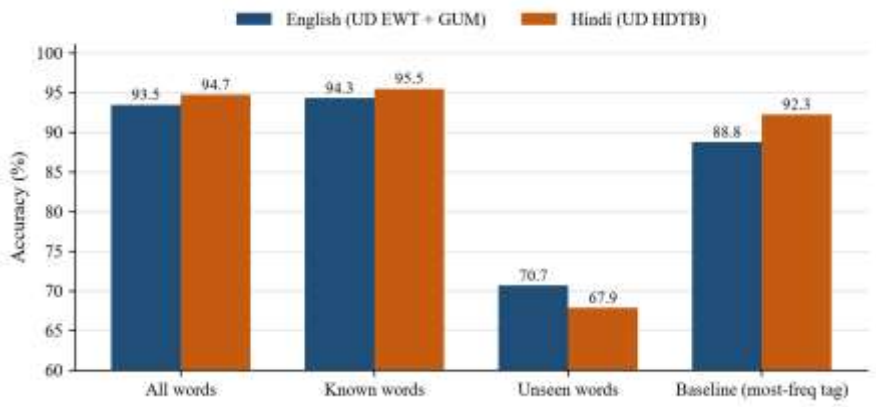


Figure 4: HMM tagger accuracy, English vs. Hindi

CONCLUSION:

N-Ticipate shows that a classical statistical pipeline, a pruned trigram model with stupid backoff, a prefix trie and an HMM tagger, is enough to build a usable, real-time, fully private autocomplete system. Implementing every component from scratch made the trade-offs concrete: Kneser-Ney wins on perplexity but loses after pruning, add-one smoothing pushes the tagger below a most-frequent-tag baseline, and log-space Viterbi is necessary rather than optional. The POS reranker adds about one point of next-word hit@1 at negligible cost, and the same tagger class works for Hindi unchanged. The most important lesson came from running the application rather than from a metric: held-out evaluation cannot reveal a domain mismatch, and swapping a 1961 corpus for modern text was what made the suggestions genuinely useful. The project is complete, with all eight phases implemented, measured and covered by 609 automated tests.

REFERENCES:

[1] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. draft, Chapters 3 (N-gram Language Models) and 8 (Sequence Labeling for Parts of Speech), 2024.

[2] R. Kneser and H. Ney, "Improved backing-off for M-gram language modeling," Proc. IEEE ICASSP, pp. 181–184, 1995.

[3] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling," Computer Speech & Language, vol. 13, no. 4, pp. 359–394, 1999.

[4] T. Brants, A. C. Popat, P. Xu, F. J. Och and J. Dean, "Large language models in machine translation," Proc. EMNLP-CoNLL, pp. 858–867, 2007 (stupid backoff).

[5] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, 1989.

[6] G. Penedo et al., "The FineWeb datasets: decanting the web for the finest text data at scale," NeurIPS Datasets and Benchmarks, 2024.

[7] Y. Chen, Y. Liu, L. Chen and Y. Zhang, "DialogSum: A real-life scenario dialogue summarization dataset," Findings of ACL-IJCNLP, 2021.

[8] J. Nivre et al., "Universal Dependencies v2," Proc. LREC, 2020, English-EWT, English-GUM and Hindi-HDTB treebanks.

[9] S. Bird, E. Klein and E. Loper, Natural Language Processing with Python, O'Reilly Media, 2009.